CS 6362 Advanced Machine Learning, Fall 2026
Assignment 2: On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima

Souvik Ghosh

October 21, 2026

1 Problem Statement and Methodology

1.1 Problem Statement

Training a network means solving

$$\min_{x\in\mathbb{R}^n} f(x) = \frac{1}{M}\sum_{i=1}^{M} f_i(x), \tag{1}$$

where x is the vector of all weights, M is the number of training points and f_i is the loss (cross-entropy here) on point i. SGD and its variants take steps

$$x_{k+1} = x_k - \alpha_k \frac{1}{|B_k|}\sum_{i\in B_k} \nabla f_i(x_k), \tag{2}$$

with a random batch $B_k \subset \{1,\dots,M\}$ of fixed size and step size α_k. A bigger $|B_k|$ would make training easier to parallelize, but in practice it hurts test accuracy even when the training accuracy is about the same. The paper asks why this happens. It compares a small-batch (SB) regime, $|B_k| = 256$, with a large-batch (LB) regime, $|B_k| = 0.1M$, both trained with ADAM.

1.2 Model and Methodology

The main claim is that LB methods converge to *sharp* minimizers, where $\nabla^2 f(x)$ has many large positive eigenvalues, while SB methods converge to *flat* ones. Sharp minima are said to generalize worse. The authors also list other possible explanations (overfitting, saddle points, LB staying close to the initial point) and argue that the data supports only the last one plus the sharp/flat picture.

They test this on six networks (fully connected and convolutional, on MNIST, TIMIT, CIFAR10 and CIFAR100) using three kinds of evidence:

1. **1-D parametric plots** (Figure 3). They take the SB solution $x_s^\star$ and the LB solution $x_\ell^\star$ and evaluate $f(\alpha x_\ell^\star + (1-\alpha)x_s^\star)$ for $\alpha \in [-1, 2]$ on the train and test sets, along with the accuracy. Around $\alpha = 1$ the curve shoots up much faster than around $\alpha = 0$.

2. **A sharpness metric** that measures the largest loss increase inside a small box around the solution (below).

3. **Warm-starting and distance experiments.** LB runs started from SB iterates after enough epochs end up flat and generalize well, and $\|x_s^\star - x_0\|/\|x_\ell^\star - x_0\|$ is about 3–10, so SB moves much farther from the initialization.

1.3 Mathematical Derivations and Intuition

Near a minimizer $x^\star$, a second-order Taylor expansion gives

$$f(x^\star + \delta) \approx f(x^\star) + \tfrac{1}{2}\,\delta^\top \nabla^2 f(x^\star)\,\delta, \tag{3}$$

because the gradient is zero. If the test loss is roughly the training loss shifted by some small δ, then a large curvature turns that small shift into a big loss increase. That is the basic intuition for why sharp minima generalize worse (Figure 1 of the paper).

Computing the eigenvalues of $\nabla^2 f$ is too expensive for these networks, so they use a proxy. Let $A \in \mathbb{R}^{n\times p}$ be a matrix (either I_n or random with $p = 100$ columns) and define the box

$$\mathcal{C}_\epsilon = \left\{ z \in \mathbb{R}^p : |z_i| \le \epsilon\big(|(A^+ x)_i| + 1\big) \right\}. \tag{4}$$

The sharpness is then

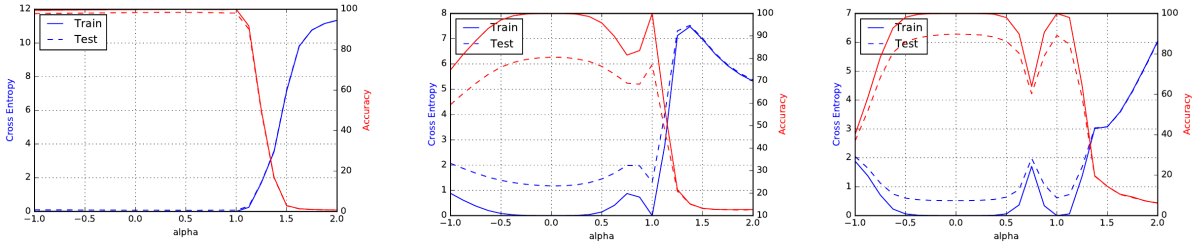
$$\phi_{x,f}(\epsilon, A) = \frac{\max_{y\in\mathcal{C}_\epsilon} f(x + Ay) - f(x)}{1 + f(x)} \times 100, \tag{5}$$

which they solve approximately with 10 iterations of L-BFGS-B. For small ϵ and $A = I_n$ this tracks the largest eigenvalue of the Hessian. The $|x_i|+1$ scaling makes the box size relative to each weight.

The explanation for *why* batch size matters is gradient noise. The mini-batch gradient is an unbiased estimate of ∇f, and its variance shrinks roughly like $1/|B_k|$. With small batches the noise is large enough to kick the iterate out of narrow basins, so it settles in a wide one. With large batches the noise is small, and the method goes into whatever basin is closest to the starting point, which often turns out to be sharp.

2 Reproduction of Figure 3 (F_1, C_1, C_2)

2.1 Original Figure



(a) F_1 (c) C_1 (d) C_2

Figure 1: Panels (a) F_1, (c) C_1 and (d) C_2 of Figure 3 in Keskar et al. (2017). Blue (left axis) is the cross-entropy loss and red (right axis) is the accuracy. Solid lines are the training set and dashed lines the test set. $\alpha = 0$ is the SB minimizer and $\alpha = 1$ is the LB minimizer.

2.2 LLM Details and Code

- **LLM (name and version):** Claude Opus 5.5 (claude.ai chat interface)
- **Shared conversation:**
  https://claude.ai/share/1c8f4ef7-8b90-4f82-bb84-55c4ef436ea4

- **GitHub repository:** https://github.com/souvik7701/ML-Assignment-2. The notebook `Paper_2.ipynb` reproduces the figure, and the results of the final run are included.

During Phases 1–3 the LLM was not given any existing code. In its first answer it stated that it "did not look at the authors' GitHub repository or any other implementation". The paper itself links the authors' repository (footnote on page 4), but nothing in the chat suggests that the LLM opened it. I ran all the training myself on a Colab T4 GPU.

2.3 Phase 1: Prompting the LLM with the Paper

I gave the LLM the paper PDF and asked it to reproduce panels F_1, C_1 and C_2 of Figure 3, split into pieces I could run one by one in Colab. I asked it to mount my Google Drive, keep the datasets there, and cache all results in Drive so that a disconnect would not lose finished work. I told it not to use, pull or look at any existing implementation.

It returned a notebook that builds the three networks from Appendix B, trains the SB and LB models with the paper's optimizer and batch sizes, runs the α sweep, plots the panels and compares the accuracies with Table 2.

Where the paper is silent, it made its own choices and listed them: the ADAM learning rate, a patience-based version of "trained until the loss function ceased to improve", a reading of "Max-Pool(3)" in C_1, and padding (without it, the deep C_2 would shrink the image to nothing). The SB and LB runs start from the same initial weights x_0, since Section 3 measures both distances to a single x_0. Batch-norm (BN) running statistics are not weights, so it re-estimated them on the training set at every α. It used one trial per regime instead of the paper's five. This first version (v1) also divided the pixel values by 255.

2.4 Phase 2: Reviewing the LLM Output

v1 ran without errors and already showed the qualitative result: the loss was flat around $\alpha = 0$ and rose steeply just past $\alpha = 1$. I asked for three changes before the next run:

1. remove the division by 255, because Appendix A says the image data sets are used "without any pre-processing";
2. make C_1 follow Appendix B.3 as literally as possible, without unsupported assumptions;
3. make sure the accuracy at the two endpoints of the curve equals the direct evaluation of the saved SB and LB models.

The third point was a real bug. v1 re-estimated the BN statistics at every α, including at $\alpha = 0$ and 1, so the endpoints of the curve were not the saved models (Table 1). The gap is largest for C_1, where a BN layer sits right after a dropout layer. PyTorch's running BN statistics are collected during training with dropout on, so they do not describe the network that is evaluated with dropout off. Re-estimating them gives a different model. F_1 has no dropout and barely changes. v1 also cut the C_1 SB run off at the 100-epoch cap while its loss was still falling.

	$\alpha = 0$ (SB), train / test		$\alpha = 1$ (LB), train / test	
net	sweep	saved model	sweep	saved model
F_1	99.98 / 98.76	99.98 / 98.72	100.00 / 98.38	100.00 / 98.38
C_1	99.56 / 71.19	96.63 / 68.98	99.20 / 67.30	93.95 / 64.50
C_2	99.64 / 85.66	99.08 / 84.84	99.59 / 82.25	98.65 / 81.26

Table 1: v1: accuracy (%) at the endpoints of the sweep against the direct evaluation of the saved models.

The LLM's second version (v2) fed raw pixels, read "MaxPool(3)" as a 3×3 window with stride 3, loaded the saved models unchanged at the endpoints, and interpolated the BN statistics together with the weights in between. This version failed badly (Figure 2). Outside $[0, 1]$ the curves blew up: the loss reached absurd values for F_1 and C_1, and for C_2 it became NaN just past the LB minimizer. C_1 also got worse. Stride-3 pooling shrinks its last feature map to a single pixel, and test accuracy fell even further below the paper. Neither Appendix B.3 nor the AlexNet paper it cites supports stride 3.

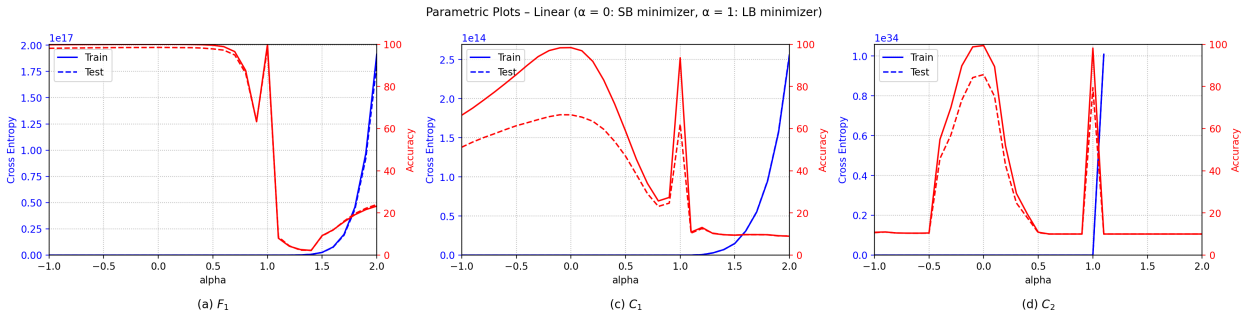


Figure 2: The failed v2 run, where the BN statistics were interpolated along with the weights. Outside $[0, 1]$ the losses explode or become NaN.

2.5 Phase 3: Good-Faith Reproduction

I reported the C_1 gap and the unstable curves back to the LLM and asked it to check the implementation against Appendix B, to handle the unstable regions, and to make it explicit that only one trial was used. I also asked it not to force the results toward the paper with arbitrary changes. The final version (v3) has these changes:

- **C_1 pooling back to 3×3 with stride 2.** This is AlexNet's overlapping pooling, which B.3 says C_1 is based on.
- **BN population statistics.** After training, each model's BN statistics are computed once over the whole training set with the final weights and dropout off. This is the inference rule of Ioffe and Szegedy, the BN paper the authors cite. The models are evaluated and saved with these statistics.
- **The same BN rule along the line.** At every interpolated α the weights are interpolated and the BN statistics are recomputed by the same rule. At $\alpha = 0$ and 1 the saved models are loaded unchanged. Two checks run after the sweep. The endpoint values must equal the saved models' evaluation, and recomputing the statistics at the endpoints must reproduce the

saved models. Both checks pass for all three networks, so the recomputation does not shift the curve.

- **Stopping rule.** A longer patience and a higher epoch cap, the same for every network and regime.
- **Display.** The loss axis is capped at about the range of the paper's own panels. Larger values are marked with ▲ on the top edge.
- **Single trial made explicit.** The figure and the Table 2 comparison state that every number comes from one run.

Every value in the v3 sweep is finite.

2.6 Phase 4: Comparison with an Existing Implementation

The authors' repository is linked in the paper:

https://github.com/keskarnitish/large-batch-training

It has Keras/Theano code that trains C_1–C_4 and draws the parametric plot, plus a later PyTorch demo on a VGG-11. The paper's footnote describes it as code for the parametric plot "on exemplary networks", so it is not necessarily the exact setup behind Figure 3.

It confirms several of the LLM's choices. C_1 pools with a 3×3 window and stride 2, as in v3 (not v2's stride 3). The SB and LB runs start from the same saved x_0, the batch sizes match, and ADAM runs at its default settings. Most usefully, it answers the BN question. Only the trainable weights are interpolated, and the Keras BN layers (mode 2) normalize with the statistics of the current batch even at evaluation time. So at every α the authors' BN statistics are in effect recomputed from the data, which is close to what v3 does. The PyTorch demo interpolates the running statistics as well, but it runs the sweep in training mode, so it also ends up using batch statistics.

There are also real discrepancies:

1. C_1 **uses stride-1 convolutions.** The paper writes $2 \times [64, 5, 5, 2]$, where the last number is defined as the stride. The code uses stride 1, so their C_1 keeps much more of the image and has about six times as many parameters as the one built from the paper's text. This is most likely the main reason my C_1 is about 10 points below Table 2 on the test set. The text and the code of the paper disagree here.

2. C_2 **has much more dropout.** The code puts dropout after most convolution layers as well as around the dense layer. B.4 says the layers are "as in B.3", which has dropout on the dense layers only. Stronger regularization probably explains part of my C_2 test gap.

3. **Inputs are divided by 255**, although Appendix A says "no pre-processing". This makes almost no difference here, because every network applies BN right after its first layer. BN removes both the scale and the mean of its input, so it cancels the $1/255$ factor.

4. **Training length.** The scripts train for a fixed number of epochs, not "until the loss function ceased to improve".

5. **Initialization.** Keras 1 uses Glorot-uniform. I use PyTorch's default uniform initialization. Both are "uniformly distributed random starting points", as the paper says.

I made no changes to the code after this comparison. Retraining C_1 with stride-1 convolutions, and C_2 with the authors' dropout layout, would be the natural next step to test items 1 and 2 directly.

2.7 Reproduced Figure

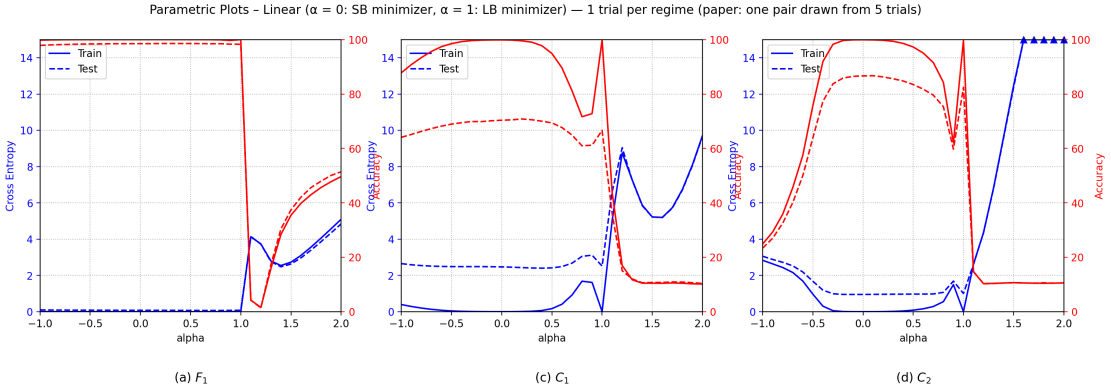


Figure 3: My reproduction (v3, one trial per regime). The loss axis is capped, and the ▲ markers for C_2 stand for losses above the cap.

The main claim is reproduced in all three panels. Around the SB minimizer the loss is flat over a wide range of α. The LB minimizer sits in a narrow dip, and the loss rises within one grid step on either side. Table 2 measures this directly from the sweep.

	SB ($\alpha = 0$)			LB ($\alpha = 1$)		
net	$\alpha = -0.1$	$\alpha = 0$	$\alpha = 0.1$	$\alpha = 0.9$	$\alpha = 1$	$\alpha = 1.1$
F_1	8.4×10^{-5}	7.0×10^{-5}	6.5×10^{-5}	0.0095	2.7×10^{-6}	4.13
C_1	0.0050	0.0046	0.0061	1.62	0.0045	5.02
C_2	0.0034	0.0023	0.0034	1.49	0.0071	2.62

Table 2: Training loss at each minimizer and 0.1 away from it along the line. Around SB it barely changes. Around LB it rises by 2–6 orders of magnitude.

Shape of each panel.

- F_1**.** As in the paper, train and test loss are near zero on all of $[-1, 1]$ and accuracy stays high. Right after $\alpha = 1$ the loss jumps and the accuracy collapses. Beyond that the curves differ. In the paper the loss keeps rising and the accuracy stays near zero. In mine the loss dips again and the accuracy recovers to about 50% at $\alpha = 2$. I have no good explanation for this far region.

- C_1**.** Both panels show the same features: a bump in the loss and a dip in accuracy just before $\alpha = 1$, a narrow dip at $\alpha = 1$, a peak just past it, a partial decrease, and accuracy near chance further out. My test loss at the SB minimizer is higher, because my C_1 generalizes worse (below).

- C_2**.** Again the same bump before $\alpha = 1$ and the narrow dip at $\alpha = 1$, and in both the accuracy collapses past $\alpha \approx 1.2$. My loss grows much faster beyond the LB minimizer (about 30 at $\alpha = 2$ against about 6).

Accuracies. Table 3 compares the final models with Table 2. All networks fit the training set, and in every case LB generalizes worse than SB. For F_1 the SB–LB test gap is the same as in the paper (0.2 points). For C_1 and C_2 my gap is larger (about 4 points against 2 to 3), and both networks are below the paper in absolute test accuracy.

net		train SB	train LB	test SB	test LB
F_1	ours	100.00	100.00	98.53	98.31
	paper	99.66 ± 0.05	99.92 ± 0.01	98.03 ± 0.07	97.81 ± 0.07
C_1	ours	99.90	99.91	70.38	66.64
	paper	99.89 ± 0.02	99.66 ± 0.20	80.04 ± 0.12	77.26 ± 0.42
C_2	ours	99.94	99.77	86.67	82.47
	paper	99.99 ± 0.04	99.99 ± 0.01	89.24 ± 0.12	87.26 ± 0.07

Table 3: Accuracy (%) of the final models. Ours are single runs. The paper's are mean ± standard deviation over 5 runs.

2.8 Failures, Diagnosis, and Corrections

Why interpolating BN statistics fails. At evaluation time a BN layer computes

$$\mathrm{BN}(z) = \gamma \, \frac{z - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \tag{6}$$

where μ and σ^2 should be the mean and variance of the pre-activation z *under the current weights.* They are not free parameters but functions of the weights, $\mu(x)$ and $\sigma^2(x)$. When v2 interpolated them linearly,

$$\sigma^2_\alpha = \sigma^2_s + \alpha \, (\sigma^2_\ell - \sigma^2_s), \tag{7}$$

the result is negative whenever $\alpha > \sigma^2_s/(\sigma^2_s - \sigma^2_\ell)$ for a channel with $\sigma^2_s > \sigma^2_\ell$ (and similarly for $\alpha < 0$). It can also simply get close to zero. Then $1/\sqrt{\sigma^2 + \epsilon}$ is huge or undefined. Even when it stays positive, the stored $\mu_\alpha, \sigma^2_\alpha$ do not match the real statistics of z at the interpolated weights. Each layer then passes on activations with the wrong scale, and the error multiplies through the depth. That is why the deep C_2 failed first and worst (NaN). Recomputing $\mu(x_\alpha)$ and $\sigma^2(x_\alpha)$ from the data defines the network at x_α properly, and it is also what the authors' code effectively does.

The remaining C_1 gap. The C_1 test accuracy is about 10 points below Table 2 in both regimes, while training accuracy matches. The paper leaves several things open (learning rate, initialization, exact stopping point), but Phase 4 points to one large concrete cause. The authors' C_1 uses stride-1 convolutions and has about 6× more parameters than the network described in B.3. With stride 2 in the convolution and stride 2 in the pooling, my C_1 reduces the image to 2×2 after only two blocks and loses most of the spatial information. I followed the paper's text, so this mismatch is most likely an inconsistency between the paper and the code behind it.

The C_2 gap. My C_2 is a few points below Table 2. The authors' C_2 has dropout after most convolution layers, which the paper does not mention. With far less regularization my network overfits more, especially the LB model, which stops early.

Single-trial variability. The paper's Table 2 is a mean over 5 runs, and Figure 3 uses one pair drawn from them. I have one run per regime. Near the end of training, the LB test accuracy of C_1 and C_2 still moved by a few points from epoch to epoch, so a difference of that size on a single LB run means little by itself. The SB runs were much steadier.

Stopping rule. F_1 LB ran to the epoch cap. Its training loss keeps shrinking toward zero by a roughly constant factor, so in relative terms it always "improves" and the rule never fires. The model itself had long since converged, with 100% training accuracy and a flat test accuracy.

Distance from the starting point. Section 3 of the paper says $\|x_s^\star - x_0\|/\|x_\ell^\star - x_0\|$ was 3–10. I get 17.4 for F_1, 4.3 for C_1 and 14.0 for C_2. So SB moves much farther from x_0 than LB in every case, as the paper says, but for F_1 and C_2 the ratio is larger than the paper's range. One possible reason is that my SB runs train much longer than the authors' script, and they may keep moving away from x_0 after the loss is already small.

2.9 Required Questions

Which networks and datasets correspond to panels F_1, C_1, C_2?

- F_1: a fully connected network on MNIST (B.1): a 784-dimensional input, 5 batch-normalized layers of 512 ReLU units, and a 10-way softmax output.
- C_1: a shallow convolutional network on CIFAR-10 (B.3), a modified AlexNet: 2× [64 filters, 5×5, stride 2]–MaxPool(3), then dense layers of 384 and 192 units with dropout (retention 0.5), and a 10-way output. BN on all layers, ReLU activations.
- C_2: a deep convolutional network on CIFAR-10 (B.4), a modified VGG: stacks of 2×64, 2×128, 3×256, 3×512, 3×512 filters (3×3, stride 1), each followed by MaxPool(2), then a 512-unit dense layer and a 10-way output. Layers are as in B.3. (B.4 calls this "the C3 network", which is a typo for C_2.)

What range of α does Figure 3 use, and what quantities are plotted on each axis? $\alpha \in [-1, 2]$, on the horizontal axis. The left vertical axis (blue) is the cross-entropy loss $f(\alpha x_\ell^\star + (1-\alpha)x_s^\star)$. The right vertical axis (red) is the classification accuracy at the same point. Solid lines are the training set and dashed lines the test set. $\alpha = 0$ is the SB minimizer, $\alpha = 1$ the LB minimizer, and values outside $[0, 1]$ extrapolate past each one.

What batch sizes and optimizer define the SB and LB runs? Both use ADAM. SB uses batches of 256 points. LB uses 10% of the training set: 6000 for MNIST and 5000 for CIFAR-10. The paper does not give ADAM's learning rate or other settings.

2.10 Implementation Choices

- **BN statistics at interpolated weights.** Recomputed at every α: the weights (including BN's γ and β) are interpolated, then the BN mean and variance are computed over the whole training set with dropout off. The saved SB and LB models use the same rule, so the curve has one definition everywhere and its endpoints are the saved models themselves.

- **Shared initialization.** Yes. The SB and LB runs start from the same x_0 (same seed) but see different batch orders. This makes the $\|x^\star - x_0\|$ comparison meaningful, and it is also what the authors' code does.

- **Other choices.** ADAM with learning rate 10^{-3}; raw pixel values as input; padding $\lfloor k/2 \rfloor$; C_1 pooling 3×3 with stride 2; PyTorch default initialization; stopping after 20 epochs without a relative improvement of 10^{-3} in the training loss (cap 300); fixed-size random batches, with the remainder of each epoch dropped; mixed precision for training and fp32 for evaluation; an α grid of step 0.1 (31 points); one trial per regime.

3 Critical Review

3.1 Strengths

The paper takes a problem that practitioners already knew about and gives a clear, testable explanation for it. The evidence is varied: parametric plots, a sharpness number, batch-size sweeps and warm-starting. These all point the same way across six networks, and Table 2 reports means and standard deviations over 5 runs. The warm-starting result is also useful in practice, because it suggests gradually increasing the batch size as a fix. The authors are honest that the sharpness metric is a heuristic and that they only provide numerical evidence, not a proof.

3.2 Weaknesses

The main issue is that the paper shows a correlation between batch size, sharpness and test accuracy, but not that sharpness *causes* the gap. The sharpness metric is also not invariant to reparametrization. For ReLU networks you can scale one layer up and the next layer down without changing the function, which makes a minimum look as sharp or as flat as you like [2]. So a flat-vs-sharp comparison based on raw weights is fragile.

The LB setup is also not tuned. The same optimizer settings are used for both regimes, and the LB run makes far fewer updates per epoch. Later work showed that much of the gap closes when the learning rate is scaled with the batch size and LB is trained for more iterations [3, 4]. So part of the gap may come from the optimization budget rather than from the batch size itself.

The 1-D plots only look at one line through weight space. Figure 3 also shows only one randomly picked SB/LB pair, with no sense of how much it varies between runs.

3.3 Potential Errors

- "Train until the loss stops improving" is vague. The final solutions, and therefore the plots, depend on this stopping rule, but it is not stated precisely.

- The maximization in the sharpness metric uses only 10 L-BFGS-B iterations. This gives a lower bound on the true maximum, and it could be looser for some solutions than others.

- Since $x_\ell^\star$ is much closer to x_0 than $x_s^\star$, the two solutions have different scales. In the linear interpolation, a step of size $\Delta\alpha$ is the same absolute distance on both sides, but a very different distance relative to each solution's norm. This could make the LB side look sharper than it is.

- For networks with batch norm, the loss at interpolated weights also depends on how the BN statistics are computed. The paper does not say.

3.4 Comparison with OpenReview Reviews

The reviews were positive: 8, 10 and 6. The decision note said all reviews, including the public one, were "extremely positive". The reviews were also short. Reviewer 1 wrote a single sentence saying the paper is useful to the community. Reviewer 2 called it "little novelty but valuable empirical evidence", which I agree with. Reviewer 3 called it a "good paper" and asked for more experiments, for example whether adding rescaled Gaussian noise to the LB gradients would recover the SB behavior. A public comment by Alex Lamb asked similar questions.

The closest any reviewer came to my main criticism was Reviewer 2. They doubted the original claim that the sharpness metric is scale-invariant, and the authors removed that claim in the revision. That is the same issue as the reparametrization problem I raised, but the review stopped there. Dinh et al. [2] made the full argument later.

None of the reviewers questioned whether the LB runs were tuned fairly, the single SB/LB pair in Figure 3, the BN statistics at interpolated weights, or the vague stopping rule. So I think the reviews were much less critical than mine. They mostly accepted the sharp-minima explanation and asked for more experiments, while I question whether the evidence supports sharpness as the *cause* of the gap.

References

[1] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang. On large-batch training for deep learning: Generalization gap and sharp minima. In *International Conference on Learning Representations*, 2017.

[2] L. Dinh, R. Pascanu, S. Bengio, and Y. Bengio. Sharp minima can generalize for deep nets. In *International Conference on Machine Learning*, 2017.

[3] E. Hoffer, I. Hubara, and D. Soudry. Train longer, generalize better: closing the generalization gap in large batch training of neural networks. In *Advances in Neural Information Processing Systems*, 2017.

[4] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He. Accurate, large minibatch SGD: Training ImageNet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017.